virtual onscreen keyboard to provide ready access to characters such as the @ sign when entering an email address.

Although the support for these functions varies by browser, those that don't provide extra support for these functions simply display a plain, unadorned input box. Once again, nothing is gained by waiting. If you have an input field that's expected to contain an email address, don't simply use `text_field` —go ahead and start using `email_field` now.

Lines 14, 24, and 34 demonstrate three different ways to provide a constrained set of options. Although the display may vary a bit from browser to browser, these approaches are all well supported across all browsers. The `select` method is particularly flexible—it can be passed an `Enumeration` as shown here, an array of pairs of name-value pairs, or a `Hash`. A number of form options helpers[161] are available to produce such lists from various sources, including the database.

Finally, lines 39 and 44 show prompts for a date and time, respectively. As you might expect by now, Rails provides plenty of options here too.[162]

Not shown in this example are `hidden_field` and `password_field`. A hidden field isn't displayed at all, but the value is passed back to the server. This may be useful as an alternative to storing transient data in sessions, enabling data from one request to be passed onto the next. Password fields are displayed, but the text entered in them is obscured.

This is more than an adequate starter set for most needs. Should you find that you have additional needs, you're likely to find a helper or gem is already available for you. A good place to start is with the Rails Guides.[163]

Meanwhile, let's explore how the data form's submit is processed.